1

HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

C# ve .NET'e Giriş & Temel Dil Özellikleri

Hazırlık ve Temel Atma

Bu haftanın kazanımları

1. **.NET** platformunun mimarisini ve .NET 10 sürümünün getirdiklerini açıklayabilir.
2. Visual Studio 2026 üzerinde bir **console application** oluşturup çalıştırabilir ve hata ayıklayabilir.
3. **Value type** ve **reference type** ayrımını yaparak doğru veri tipini seçebilir.
4. **if / switch / for / while / foreach** yapılarıyla program akışını yönetebilir.
5. **Array**, **List<T>** ve **Dictionary<TKey,TValue>** arasında bilinçli seçim yapabilir.

İçindekiler

1 **.NET Ekosistemi ve .NET 10'a Genel Bakış**
Runtime, BCL, SDK · Derleme süreci · Garbage Collector · Sürüm tarihçesi · Uygulama modelleri · .NET CLI

2 **Visual Studio IDE ve Temel Kullanım**
Sürümler ve kurulum · Solution / Project · Temel pencereler · Debugging · Kısayollar

3 **Değişkenler, Veri Tipleri ve Operatörler**
Value / reference type · Temel tipler · var, const, readonly · Nullable · Tip dönüşümleri · String işlemleri

4 **Kontrol Yapıları**
if / else · switch statement ve expression · for, while, do-while, foreach · break ve continue

5 **Diziler ve Koleksiyonlar**
Array · Çok boyutlu ve jagged diziler · List<T> · Dictionary<K,V> · HashSet, Queue, Stack

★ **Haftanın Özeti, Sözlük, Alıştırmalar ve Ekstra Okuma**
7 maddelik özet · 12 terimlik sözlük · 7 alıştırma · 9 kaynak bağlantısı

1. .NET Ekosistemi ve .NET 10'a Genel Bakış

Bu bölümün anahtar kelimeleri

`platform` (platform) · `runtime` (çalışma zamanı) · `SDK` (yazılım geliştirme kiti) · `compiler` (derleyici) · `garbage collector` (çöp toplayıcı) · `cross-platform` (çapraz platform) · `LTS` (uzun dönem destek)

1.1 .NET nedir?

.NET, Microsoft tarafından geliştirilen **açık kaynaklı** ve **cross-platform (çapraz platform)** bir uygulama geliştirme platformudur. Tek bir dil değildir; birden fazla dili (C#, F#, Visual Basic) barındıran bir **ecosystem (ekosistem)**'dir. Windows, Linux ve macOS üzerinde aynı kaynak kod ile çalışan uygulamalar üretebilirsiniz.

Bir .NET kurulumu üç ana parçadan oluşur:

- **Runtime (çalışma zamanı)** — Derlenmiş kodu çalıştıran motor. Adı **CLR** (Common Language Runtime / Ortak Dil Çalışma Zamanı)'dır.
- **BCL (Base Class Library / Temel Sınıf Kütüphanesi)** — `String`, `List<T>`, `File`, `HttpClient` gibi hazır binlerce sınıf.
- **SDK (Software Development Kit)** — Derleyici (**Roslyn**), `dotnet` komut satırı aracı ve proje şablonları. Sadece uygulama çalıştıracaksanız Runtime yeterlidir; **geliştirme** yapacaksanız SDK kurmalısınız.

1.2 Kaynak koddan çalışan programa: derleme süreci

C# kodu doğrudan makine diline çevrilmez. Aradaki iki aşama .NET'in taşınabilirliğinin ve performansının temelidir:

Tablo 1.1 — Derleme zinciri

Aşama	Ne olur?	Sonuç
Compile time (derleme zamanı)	Roslyn derleyicisi `.cs` dosyalarını okur, sözdizimi ve tip hatalarını yakalar.	**IL** (Intermediate Language) içeren `.dll` / `.exe` dosyası
Run time (çalışma zamanı)	**JIT** (Just-In-Time) derleyicisi, IL kodunu o an çalışılan işlemcinin makine diline çevirir.	İşlemciye özel, optimize edilmiş native kod
AOT (isteğe bağlı)	Ahead-Of-Time: IL, daha yayın öncesinde native koda çevrilir.	Anında başlayan, küçük boyutlu uygulama

> **[i] Neden ara dil (IL)?**
> Aynı IL dosyası hem Windows'ta hem Linux'ta, hem x64 hem ARM işlemcide çalışabilir. Platform farkını JIT derleyicisi çalışma anında kapatır. **"Bir kez derle, her yerde çalıştır"** yaklaşımının .NET'teki karşılığı budur.

1.3 Bellek yönetimi: Garbage Collector

C ve C++'ta ayırdığınız belleği elle geri vermek zorundasınızdır. C#'ta bu işi **GC (Garbage Collector / Çöp Toplayıcı)** yapar: artık hiçbir değişkenin işaret etmediği nesneleri tespit eder ve belleği serbest bırakır. Bu, **memory leak (bellek sızıntısı)** hatalarının büyük bölümünü kendiliğinden ortadan kaldırır.

1.4 .NET'in tarihçesi ve bugünü

Tablo 1.2 — .NET sürüm ailesi

Ürün	Yıl	Durum
.NET Framework (1.0 – 4.8)	2002 – 2019	Yalnızca Windows. Yeni proje açılmaz, sadece bakım.
.NET Core (1.0 – 3.1)	2016 – 2019	İlk çapraz platform sürüm. Desteği bitti.
.NET 5 / 6 / 7 / 8 / 9	2020 – 2024	Birleşik tek platform. Çift numaralar LTS.
.NET 10	**11 Kasım 2025**	**Güncel LTS.** Kasım 2028'e kadar desteklenir.

.NET 10, C# 14 ve Visual Studio 2026 ile birlikte yayımlandı. Bu ders boyunca **.NET 10 (LTS)** hedef alınacaktır. LTS (Long Term Support) sürümler üç yıl güvenlik ve hata düzeltmesi aldığı için ticari projelerde tercih edilir.

.NET 10 ve C# 14 ile gelen öne çıkan yenilikler

- **File-based apps** — Proje dosyası (`.csproj`) olmadan tek bir `.cs` dosyasını `dotnet run app.cs` ile çalıştırabilirsiniz. Derste küçük denemeler için çok pratiktir.
- **Extension members** — Var olan tiplere yalnızca metot değil, artık **property** ve **operator** da eklenebiliyor.
- **`field` keyword** — Property içinde arka plan alanına (backing field) elle isim vermeye gerek kalmadı. (3. haftada Encapsulation konusunda göreceğiz.)
- **Null-conditional assignment** — `obj?.Prop = value;` yazımı artık geçerli.
- **Runtime iyileştirmeleri** — Daha iyi JIT inlining, daha düşük GC duraklaması, gelişmiş Native AOT.

1.5 Uygulama modelleri

Tablo 1.3 — .NET ile ne geliştirilebilir?

Model	Kullanım alanı	Şablon komutu
Console	Komut satırı araçları, öğrenme, otomasyon	`dotnet new console`
ASP.NET Core	Web API, web sitesi, mikroservis	`dotnet new webapi`
Blazor	C# ile yazılan web arayüzü (JavaScript'e alternatif)	`dotnet new blazor`
.NET MAUI	Android, iOS, Windows, macOS mobil/masaüstü	`dotnet new maui`
WPF / WinForms	Klasik Windows masaüstü uygulamaları	`dotnet new wpf`
Class Library	Başka projelerde kullanılacak ortak kod	`dotnet new classlib`

1.6 İlk programımız

C# 9 ile gelen **top-level statements (üst düzey ifadeler)** sayesinde artık `Main` metodu yazmak zorunlu değildir. Derleyici bu kodu arka planda sizin yerinize `Main` içine sarar.

Kod 1.1 — Program.cs (modern, top-level statements)

```
// Ekrana metin yazdiran en kucuk C# programi
Console.WriteLine("Merhaba Dunya!");

Console.Write("Adiniz: ");            // alt satira gecmez
string ad = Console.ReadLine() ?? "";  // kullanicidan girdi al

Console.WriteLine($"Hos geldin, {ad}!");  // string interpolation
```

Kod 1.2 — Aynı program klasik yazımla (C# 9 öncesi / hâlâ geçerli)

```
using System;

namespace IlkUygulama
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Merhaba Dunya!");
        }
    }
}
```

> **[+] Kod 1.2'yi de anlamalısınız**
>
> İnternetteki örneklerin ve eski projelerin çoğu klasik yazımı kullanır. İki yazım tamamen eşdeğerdir; top-level statements yalnızca bir **syntactic sugar (sözdizimsel kolaylık)**'tır.

1.7 Komut satırı (.NET CLI)

Kod 1.3 — Sık kullanılan dotnet komutları

```
dotnet --version                # kurulu SDK surumunu gosterir
dotnet --list-sdks              # tum SDK'lari listeler

dotnet new console -o IlkProje  # IlkProje klasorunde console projesi olustur
cd IlkProje
dotnet build                    # derle (hatalari gosterir)
dotnet run                      # derle + calistir
dotnet run app.cs               # .NET 10: projesiz tek dosya calistirma

dotnet new sln -n Okul          # solution olustur
dotnet sln add IlkProje         # projeyi solution'a ekle
```

2. Visual Studio IDE ve Temel Kullanım

Bu bölümün anahtar kelimeleri

`IDE` (tümleşik geliştirme ortamı) · `solution` (çözüm) · `project` (proje) · `build` (derleme) · `debug` (hata ayıklama) · `breakpoint` (kesme noktası) · `IntelliSense` (akıllı kod tamamlama)

2.1 Sürümler ve kurulum

Visual Studio 2026, 11 Kasım 2025'te .NET 10 ile birlikte yayımlandı. Üç ana sürümü vardır:

- **Community — Ücretsiz.** Öğrenciler, bireysel geliştiriciler ve açık kaynak projeleri için. Bu derste kullanacağımız sürüm budur.
- **Professional** — Küçük ekipler için ücretli sürüm.
- **Enterprise** — Büyük kurumlar; gelişmiş test ve mimari araçları.

Kurulumda **workload (iş yükü)** seçmeniz istenir. Bu ders için gereken minimum seçim:

Kod 2.1 — Kurulumda işaretlenecek workload'lar

```
[x] .NET desktop development     -> Console, WinForms, WPF projeleri
[x] ASP.NET and web development  -> Web API, Blazor (ileriki haftalar)
[ ] Data storage and processing  -> SQL Server, EF Core araclari (opsiyonel)
```

> **[i] Visual Studio vs Visual Studio Code**
>
> İkisi farklı ürünlerdir. **Visual Studio** tam donanımlı bir IDE'dir (yalnızca Windows/Mac). **VS Code** ise hafif bir metin editörüdür; C# geliştirmek için **C# Dev Kit** eklentisi kurulmalıdır. Linux kullanıyorsanız VS Code + C# Dev Kit ya da JetBrains Rider tercih edin; bu dersteki tüm kodlar her üçünde de aynı şekilde çalışır.

2.2 Solution ve Project kavramı

Visual Studio'da en üst kapsayıcı **solution (çözüm, `.sln`)**'dur. Bir solution içinde birden çok **project (proje, `.csproj`)** bulunabilir. Her proje derlendiğinde kendi `.dll` veya `.exe` çıktısını üretir.

Kod 2.2 — Tipik bir klasör yapısı

```
Okul.sln                    <- Solution: butun projeleri bir arada tutar
+-- Okul.ConsoleApp/        <- Proje 1: calistirilabilir uygulama
|   +-- Okul.ConsoleApp.csproj
|   +-- Program.cs
+-- Okul.Core/              <- Proje 2: ortak siniflar (class library)
    +-- Okul.Core.csproj
    +-- Ogrenci.cs
```

2.3 Arayüzün temel pencereleri

Tablo 2.1 — Mutlaka tanımanız gereken pencereler

Pencere	Görevi
Solution Explorer	Proje ve dosya ağacı. Yeni sınıf eklemek buradan yapılır.
Editor	Kod yazdığınız alan. **IntelliSense** burada otomatik tamamlama sunar.
Error List	Derleme hataları (Error) ve uyarılar (Warning) listesi.
Output	Derleyicinin ürettiği ayrıntılı çıktı.
Locals / Watch	Hata ayıklama sırasında değişkenlerin anlık değerleri.
Call Stack	O anda hangi metodun hangi metottan çağrıldığı zinciri.

2.4 Debugging — hata ayıklama

Bir program "çalışmıyor" demek yetmez; **nerede** ve **neden** yanlış davrandığını görmeniz gerekir. **Breakpoint (kesme noktası)**, programı belirli bir satırda dondurup o andaki tüm değişken değerlerini incelemenizi sağlar. Satır numarasının solundaki gri şeride tıklayarak (veya **F9**) breakpoint koyabilirsiniz.

Tablo 2.2 — Hayat kurtaran kısayollar

Kısayol	İşlev
`F5`	Hata ayıklayarak çalıştır / bir sonraki breakpoint'e devam et
`Ctrl + F5`	Hata ayıklamadan çalıştır (daha hızlı)
`F9`	Breakpoint koy / kaldır
`F10`	**Step Over** — satırı çalıştır, metodun içine girme
`F11`	**Step Into** — çağrılan metodun içine gir
`Shift + F11`	**Step Out** — içinde bulunduğun metottan çık
`Ctrl + K, Ctrl + D`	Kodu otomatik biçimlendir (girintileri düzelt)
`Ctrl + .` (nokta)	Quick Actions — hata düzeltme ve yeniden adlandırma önerileri
`F12`	Go to Definition — imlecin üzerindeki tipin tanımına git
`Ctrl + Shift + B`	Build — solution'ı derle

> **[!] Sık yapılan hata**
>
> `Console.WriteLine` ile hata aramak ("printf debugging") küçük programlarda işe yarar ama hızla kontrolden çıkar. **Breakpoint kullanmayı en baştan alışkanlık hâline getirin**; ilerideki haftalarda nesne grafiği karmaşıklaştığında tek makul yöntem odur.

3. Değişkenler, Veri Tipleri ve Operatörler

Bu bölümün anahtar kelimeleri

variable (değişken) · **data type** (veri tipi) · **value type** (değer tipi) · **reference type** (referans tipi) · **stack** (yığın) · **heap** (öbek) · **declaration** (bildirim) · **assignment** (atama) · **casting** (tip dönüşümü) · **operator** (operatör) · **literal** (sabit değer) · **scope** (kapsam)

3.1 Değişken nedir?

Variable (değişken), bellekte ayrılmış ve bir **isim** ile eriştiğimiz veri saklama alanıdır. C# **statically typed (statik tipli)** bir dildir: bir değişkenin tipi derleme zamanında bellidir ve sonradan değişmez. Bu, hataların programı çalıştırmadan önce yakalanmasını sağlar.

Kod 3.1 — Bildirim (declaration) ve atama (assignment)

```
int yas;                // declaration - sadece bildirim
yas = 21;               // assignment  - deger atama

int puan = 85;          // bildirim + atama tek satirda
double ortalama = 78.5;
string ad = "Zeynep";
bool gectiMi = true;
char harf = 'A';        // tek tirnak: char, cift tirnak: string

int a = 1, b = 2, c = 3;  // ayni tipte coklu bildirim
```

İsimlendirme (naming) kuralları ve gelenekleri

Kural / Gelenek	Örnek
Harf veya _ ile başlar; rakamla başlayamaz	`ad2` geçerli, `2ad` geçersiz
Boşluk ve Türkçe karakter kullanmayın	`ogrenciAdi` (`öğrenciAdı` değil)
Anahtar kelime olamaz	`class`, `int` kullanılamaz
Büyük/küçük harf duyarlıdır (case-sensitive)	`yas` ile `Yas` farklıdır
Yerel değişken ve parametre: **camelCase**	`ogrenciNotu`, `toplamTutar`
Sınıf, metot, property: **PascalCase**	`OgrenciBilgisi`, `HesaplaOrtalama`
private alan: **_camelCase**	`_bakiye`, `_ogrenciSayisi`
Sabit (const): **PascalCase**	`MaksimumNot`, `PiSayisi`

3.2 Value type ve reference type ayrımı

Bu, C#'ın **en kritik** kavramlarından biridir ve ilerideki haftalarda nesnelerin davranışını anlamanın anahtarıdır.

Tablo 3.1 — İki temel tip ailesi

	Value type (değer tipi)	Reference type (referans tipi)
Neyi tutar?	**Verinin kendisini**	Verinin bellekteki **adresini**
Nerede durur?	Genellikle **stack** (yığın)	Veri **heap** (öbek)'te, adres stack'te
Kopyalanınca	Bağımsız bir kopya oluşur	İki değişken **aynı** nesneyi gösterir
`null` olabilir mi?	Hayır (`int?` yazılmadıkça)	Evet
Örnekler	`int, double, bool, char, decimal, struct, enum`	`string, object, array, class, List<T>`

Kod 3.2 — Farkı gösteren deney

```
// --- VALUE TYPE ---
int x = 10;
int y = x;        // x'in DEGERI kopyalanir
y = 99;
Console.WriteLine(x);   // 10  -> x etkilenmedi

// --- REFERENCE TYPE ---
int[] dizi1 = { 1, 2, 3 };
int[] dizi2 = dizi1;    // ADRES kopyalanir, dizi degil!
dizi2[0] = 99;
Console.WriteLine(dizi1[0]);  // 99 -> ayni diziyi gosteriyorlar
```

Kod 3.2 çıktısı

```
10
99
```

> **[!] string özel bir durumdur**
>
> `string` bir reference type'tır **ama immutable (değiştirilemez)**dir. Bir string'i "değiştirdiğinizde" aslında bellekte yeni bir string oluşur, eskisi GC'ye bırakılır. Bu yüzden `string` çoğu zaman value type gibi davranır. Döngü içinde binlerce kez string birleştirmek gerekiyorsa `StringBuilder` kullanın.

3.3 Temel veri tipleri

Tablo 3.2 — En çok kullanılan built-in tipler

C# tipi	.NET karşılığı	Boyut	Aralık / Not
`byte`	System.Byte	1 byte	0 … 255
`int`	System.Int32	4 byte	±2,1 milyar — **varsayılan tam sayı**
`long`	System.Int64	8 byte	±9,2 trilyon × milyar
`float`	System.Single	4 byte	~7 basamak hassasiyet, sonek `f`

C# tipi	.NET karşılığı	Boyut	Aralık / Not
`double`	System.Double	8 byte	~15 basamak — **varsayılan ondalık**
`decimal`	System.Decimal	16 byte	28 basamak, **para için**, sonek `m`
`bool`	System.Boolean	1 byte	`true` / `false`
`char`	System.Char	2 byte	Tek Unicode karakter, tek tırnak
`string`	System.String	değişken	Metin. Reference type, immutable.

[x] Para hesaplarında asla double kullanmayın

`double` ikilik tabanda çalıştığı için `0.1 + 0.2` işlemi tam olarak `0.3` vermez. Fiyat, maaş, faiz gibi hesaplarda **her zaman** `decimal` kullanın.

Kod 3.3 — Kayan nokta tuzağı

```
double d = 0.1 + 0.2;
Console.WriteLine(d);              // 0.30000000000000004
Console.WriteLine(d == 0.3);       // False  (!)

decimal m = 0.1m + 0.2m;
Console.WriteLine(m);              // 0.3
Console.WriteLine(m == 0.3m);      // True
```

3.4 var, const ve readonly

Kod 3.4

```
var sayi = 42;             // derleyici tipi cikarir -> int
var isim = "Ali";          // -> string
var liste = new List<int>();  // -> List<int>, yazim kisalir

// var YERINE gecmez: tip yine de statiktir
// sayi = "metin";         // HATA: int'e string atanamaz

const double Pi = 3.14159;    // derleme zamani sabiti, degismez
// Pi = 3;                    // HATA

// readonly: SINIF ICINDE tanimlanir. Sadece bildirimde veya constructor
// icinde deger alabilir; sonrasinda degistirilemez.
class Ayarlar
{
    public readonly DateTime OlusturmaTarihi = DateTime.Now;
}
```

[+] var ne zaman kullanılmalı?

Tip sağ taraftan **açıkça anlaşılıyorsa** kullanın (`var musteri = new Musteri();`). Anlaşılmıyorsa tipi açıkça yazın (`int toplam = Hesapla();`) — kodu okuyan kişi metodun ne döndürdüğünü bilmek zorunda kalmasın.

3.5 Nullable tipler ve null güvenliği

Kod 3.5

```
int? yas = null;           // int? = Nullable<int>, null alabilir
string? aciklama = null;   // nullable reference type

if (yas.HasValue)
    Console.WriteLine(yas.Value);

int kesinYas = yas ?? 0;        // ?? null-coalescing: null ise 0 kullan
aciklama ??= "Aciklama yok";    // ??= null ise ata

Console.WriteLine(aciklama.Length);     // guvenli: artik null degil
Console.WriteLine(aciklama?.Length);    // ?. null ise Length'e hic gitme
```

3.6 Tip dönüşümleri (type conversion)

Kod 3.6 — Dört yöntem

```
// 1) Implicit (ortuk) - veri kaybi yoksa otomatik
int kucuk = 100;
long buyuk = kucuk;        // int -> long, sorunsuz

// 2) Explicit (acik) cast - veri kaybi riski var, siz onayliyorsunuz
double ondalik = 9.87;
int tam = (int)ondalik;    // 9  (yuvarlamaz, keser!)

// 3) Convert sinifi - null'u 0 kabul eder
int n1 = Convert.ToInt32("123");

// 4) Parse / TryParse - metinden sayiya
int n2 = int.Parse("456");             // hatali metinde EXCEPTION firlatir

if (int.TryParse("abc", out int n3))   // guvenli yontem
    Console.WriteLine(n3);
else
    Console.WriteLine("Gecerli bir sayi degil");   // buraya duser
```

> **[+] Kullanıcı girdisinde daima TryParse**
>
> `int.Parse` geçersiz metinde `FormatException` fırlatarak programı çökertir. Kullanıcıdan gelen veriyi dönüştürürken **her zaman `TryParse` kullanın**; başarı durumunu `bool` olarak döndürür, değeri `out` parametresine yazar.

3.7 Operatörler

Tablo 3.3 — Operatör grupları

Grup	Operatörler	Örnek / Açıklama
Arithmetic (aritmetik)	`+ - * / %`	`7 / 2 = 3` (int bölme!), `7 % 2 = 1` (kalan)
Assignment (atama)	`= += -= *= /= %=`	`x += 5` ≡ `x = x + 5`
Increment (artırma)	`++ --`	`i++` önce kullan sonra artır; `++i` önce artır
Comparison (karşılaştırma)	`== != < > <= >=`	Sonuç daima `bool`
Logical (mantıksal)	`&& || !`	`&&` ve `||` **short-circuit**: gerekmiyorsa sağ tarafı değerlendirmez
Null	`?? ??= ?. ?[]`	Null kontrolünü kısaltır
Ternary (üçlü)	`koşul ? a : b`	Tek satırlık if-else

Kod 3.7 — Operatör tuzakları

```
Console.WriteLine(7 / 2);        // 3   -> iki int bolunurse sonuc int!
Console.WriteLine(7 / 2.0);      // 3.5 -> bir taraf double olmali
Console.WriteLine(7.0 / 2);      // 3.5
Console.WriteLine((double)7 / 2);// 3.5

int i = 5;
Console.WriteLine(i++);          // 5  (once yazdirir, sonra 6 olur)
Console.WriteLine(i);            // 6
Console.WriteLine(++i);          // 7  (once artirir, sonra yazdirir)

int not = 72;
string sonuc = not >= 50 ? "Gecti" : "Kaldi";   // ternary
Console.WriteLine(sonuc);        // Gecti
```

3.8 String işlemleri

Kod 3.8

```
string ad = "Ayse", soyad = "Yilmaz";

// String interpolation - EN OKUNAKLI YONTEM
Console.WriteLine($"{ad} {soyad}, {2026 - 2004} yasinda.");

// Formatlama
decimal fiyat = 1234.5m;
Console.WriteLine($"{fiyat:C}");      // para birimi
Console.WriteLine($"{fiyat:N2}");     // 1.234,50
Console.WriteLine($"{0.856:P1}");     // %85,6

// Sik kullanilan metotlar
string metin = "  C# Programlama  ";
Console.WriteLine(metin.Trim());              // bastaki/sondaki bosluklari siler
Console.WriteLine(metin.ToUpper());           // BUYUK HARF
Console.WriteLine(metin.Contains("Prog"));    // True
Console.WriteLine(metin.Replace("C#", "F#"));
Console.WriteLine(metin.Trim().Split(' ')[0]);// "C#"
Console.WriteLine(string.IsNullOrWhiteSpace(metin)); // False
Console.WriteLine(string.Join(", ", "a", "b", "c")); // a, b, c
```

4. Kontrol Yapıları

Bu bölümün anahtar kelimeleri

control flow (akış denetimi) · **condition** (koşul) · **branch** (dallanma) · **loop** (döngü) · **iteration** (yineleme) · **infinite loop** (sonsuz döngü) · **pattern matching** (örüntü eşleme)

4.1 if / else if / else

Kod 4.1

```
int not = 78;

if (not >= 90)
    Console.WriteLine("AA");
else if (not >= 80)
    Console.WriteLine("BA");
else if (not >= 70)
    Console.WriteLine("BB");        // <- bu satir calisir
else if (not >= 50)
    Console.WriteLine("CC");
else
    Console.WriteLine("FF");

// Mantiksal operatorlerle bilesik kosul
int yas = 20;
bool ogrenciMi = true;
if (yas >= 18 && ogrenciMi)
    Console.WriteLine("Ogrenci indirimi uygulanir.");
```

> **[!] Süslü parantezi ihmal etmeyin**
>
> Tek satırlık gövdelerde `{ }` yazmak zorunlu değildir, ancak **her zaman yazmanız** önerilir. Sonradan ikinci bir satır eklendiğinde o satır `if`'in dışında kalır ve bulunması çok zor bir hataya yol açar.

4.2 switch statement ve switch expression

Kod 4.2 — Klasik switch statement

```
int gun = 3;

switch (gun)
{
    case 1:
        Console.WriteLine("Pazartesi");
        break;                  // break ZORUNLU
    case 6:
    case 7:                     // birden fazla case birlestirilebilir
        Console.WriteLine("Hafta sonu");
        break;
    default:
        Console.WriteLine("Hafta ici");
        break;
}
```

Kod 4.3 — switch expression (C# 8+, modern ve kısa)

```
int gun = 3;

string sonuc = gun switch
{
    1 => "Pazartesi",
    2 => "Sali",
    3 => "Carsamba",
    6 or 7 => "Hafta sonu",      // pattern matching
    _ => "Bilinmeyen"            // _ = default
};
Console.WriteLine(sonuc);

// Araliklarla pattern matching
int puan = 85;
string harf = puan switch
{
    >= 90 => "AA",
    >= 80 => "BA",
    >= 70 => "BB",
    >= 50 => "CC",
    _     => "FF"
};
Console.WriteLine(harf);       // BA
```

4.3 Döngüler (loops)

Tablo 4.1 — Hangi döngüyü seçmeli?

Döngü	Ne zaman kullanılır?
`for`	Tekrar sayısı **baştan belliyse** (1'den 100'e kadar, dizinin indeksleri...)
`while`	Koşul sağlandığı sürece; **kaç kez döneceği belli değilse**
`do-while`	Gövdenin **en az bir kez** çalışması gerekiyorsa (menü gösterimi)
`foreach`	Bir koleksiyonun **tüm elemanlarını** gezmek için — en güvenlisi

Kod 4.4 — Dört döngü, dört senaryo

```
// for: 1'den 5'e kadar
for (int i = 1; i <= 5; i++)
    Console.Write($"{i} ");                // 1 2 3 4 5

Console.WriteLine();

// while: kullanici dogru sifreyi girene kadar
int deneme = 0;
while (deneme < 3)
{
    Console.WriteLine($"{deneme + 1}. deneme");
    deneme++;
}

// do-while: menu en az bir kez gosterilir
int secim;
```

```
do
{
    Console.WriteLine("1) Ekle  2) Sil  0) Cikis");
    int.TryParse(Console.ReadLine(), out secim);
} while (secim != 0);

// foreach: koleksiyonu gez
string[] sehirler = { "Rize", "Trabzon", "Artvin" };
foreach (string sehir in sehirler)
    Console.WriteLine(sehir);
```

> **[x] foreach içinde koleksiyon değiştirilemez**
>
> `foreach` ile gezerken listeye eleman eklemek veya silmek `InvalidOperationException` fırlatır. Silmek gerekiyorsa geriye doğru bir `for` döngüsü kullanın ya da `RemoveAll` metodunu tercih edin.

4.4 break, continue ve iç içe döngüler

Kod 4.5

```
// break: dongüyü tamamen bitirir
for (int i = 1; i <= 10; i++)
{
    if (i == 5) break;
    Console.Write($"{i} ");     // 1 2 3 4
}
Console.WriteLine();

// continue: bu turu atlar, dongü devam eder
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0) continue;   // ciftleri atla
    Console.Write($"{i} ");     // 1 3 5 7 9
}
Console.WriteLine();

// Ic ice dongü: carpim tablosu
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
        Console.Write($"{i}x{j}={i * j,-4}");
    Console.WriteLine();
}
```

Kod 4.5 çıktısı (son bölüm)

```
1x1=1   1x2=2   1x3=3
2x1=2   2x2=4   2x3=6
3x1=3   3x2=6   3x3=9
```

4.5 Uygulama: asal sayı kontrolü

Kod 4.6 — Bir sayının asal olup olmadığını bulan program

```
Console.Write("Bir sayi girin: ");
if (!int.TryParse(Console.ReadLine(), out int sayi) || sayi < 2)
{
    Console.WriteLine("Gecersiz giris. 2 veya buyuk bir tam sayi girin.");
    return;
}

bool asalMi = true;
for (int bolen = 2; bolen * bolen <= sayi; bolen++)
{
    if (sayi % bolen == 0)
    {
        asalMi = false;
        break;                      // bir bolen bulduk, aramaya gerek yok
    }
}

Console.WriteLine(asalMi ? $"{sayi} asaldir." : $"{sayi} asal degildir.");
```

5. Diziler ve Koleksiyonlar

Bu bölümün anahtar kelimeleri

array (dizi) · **collection** (koleksiyon) · **index** (indis / dizin) · **element** (eleman) · **generic** (genel tip) · **key-value pair** (anahtar-değer çifti) · **fixed size** (sabit boyut) · **dynamic size** (değişken boyut)

5.1 Array (dizi)

Array, **aynı tipten** ve **sabit sayıda** elemanı bellekte ardışık olarak tutan yapıdır. Boyut bir kez belirlenir ve değiştirilemez. İndisler **0'dan başlar**.

Kod 5.1 — Dizi tanımlama ve kullanma

```
// 1) Boyut vererek olustur (tum elemanlar varsayilan deger alir: int -> 0)
int[] notlar = new int[5];
notlar[0] = 85;
notlar[1] = 90;

// 2) Dogrudan degerlerle olustur
string[] sehirler = { "Rize", "Trabzon", "Artvin", "Giresun" };

// 3) Collection expression (C# 12+) - en modern yazim
int[] sayilar = [10, 20, 30, 40, 50];

Console.WriteLine(sehirler.Length);      // 4  -> eleman sayisi
Console.WriteLine(sehirler[0]);          // Rize   (ilk eleman)
Console.WriteLine(sehirler[^1]);         // Giresun (sondan 1., C# 8+)
Console.WriteLine(string.Join(", ", sayilar[1..3]));  // 20, 30 (range)

Array.Sort(sehirler);                    // alfabetik siralar
Array.Reverse(sayilar);                  // ters cevirir
Console.WriteLine(Array.IndexOf(sehirler, "Rize"));   // kacinci indiste?
```

[x] IndexOutOfRangeException

5 elemanlı bir dizide geçerli indisler **0, 1, 2, 3, 4**'tür. `notlar[5]` yazmak `IndexOutOfRangeException` fırlatır. Yeni başlayanların en sık karşılaştığı çalışma zamanı hatasıdır: **son indis = `Length - 1`**.

Çok boyutlu ve jagged diziler

Kod 5.2

```
// Multidimensional (dikdortgen) dizi: 2 satir x 3 sutun
int[,] matris = new int[2, 3];
matris[0, 0] = 1;
matris[1, 2] = 6;

int[,] tablo = { { 1, 2, 3 }, { 4, 5, 6 } };
Console.WriteLine(tablo[1, 1]);          // 5
Console.WriteLine(tablo.GetLength(0));   // 2 -> satir sayisi

// Jagged (tirtikli) dizi: her satirin uzunlugu farkli olabilir
int[][] tirtikli = new int[3][];
tirtikli[0] = [1, 2];
tirtikli[1] = [3, 4, 5, 6];
tirtikli[2] = [7];
Console.WriteLine(tirtikli[1][3]);       // 6
```

5.2 List<T> — boyutu değişebilen dizi

`List<T>`, **generic (genel tipli)** bir koleksiyondur. Buradaki `T` tutulacak elemanın tipidir: `List<int>`, `List<string>`, `List<Ogrenci>` gibi. Generic yapı sayesinde yanlış tipte eleman eklemek **derleme zamanında** hata verir.

Kod 5.3

```
List<string> ogrenciler = new List<string>();
ogrenciler.Add("Ali");
ogrenciler.Add("Veli");
ogrenciler.AddRange(["Ayse", "Fatma"]);
ogrenciler.Insert(0, "Zeynep");          // basa ekle

ogrenciler.Remove("Veli");               // degere gore sil
ogrenciler.RemoveAt(0);                  // indise gore sil

Console.WriteLine(ogrenciler.Count);          // eleman sayisi (Length degil!)
Console.WriteLine(ogrenciler.Contains("Ali"));// True
Console.WriteLine(ogrenciler.IndexOf("Ayse"));

ogrenciler.Sort();
foreach (string ad in ogrenciler)
    Console.WriteLine(ad);

// LINQ ile hizli sorgular (ilerideki haftalarda detaylandirilacak)
List<int> notlar = [55, 90, 42, 78, 95];
Console.WriteLine(notlar.Max());                    // 95
Console.WriteLine(notlar.Average());                // 72
Console.WriteLine(notlar.Count(n => n >= 50));      // 4 -> gecenler
```

> **[i] Length mi, Count mu?**
>
> `Array` için **`Length`**, `List<T>` ve diğer koleksiyonlar için **`Count`** kullanılır. `string` için de `Length`'tir. Bu ayrım sınavlarda ve mülakatlarda sık sorulur.

5.3 Dictionary<TKey, TValue> — anahtar/değer sözlüğü

Verilere sıra numarasıyla değil, **anlamlı bir anahtarla** ulaşmak istediğinizde kullanılır. Arama işlemi eleman sayısından bağımsız olarak neredeyse sabit sürede tamamlanır.

Kod 5.4

```
Dictionary<string, int> stok = new Dictionary<string, int>
{
    ["Klavye"] = 12,
    ["Mouse"]  = 30,
    ["Monitor"] = 5
};

stok.Add("Kulaklik", 8);
stok["Mouse"] = 25;                 // var olan anahtari gunceller
stok.Remove("Monitor");

// GUVENLI OKUMA: olmayan anahtar KeyNotFoundException firlatir
if (stok.TryGetValue("Klavye", out int adet))
    Console.WriteLine($"Klavye stogu: {adet}");

if (stok.ContainsKey("Webcam"))
    Console.WriteLine("Var");
else
    Console.WriteLine("Webcam stokta yok");

foreach (KeyValuePair<string, int> kayit in stok)
    Console.WriteLine($"{kayit.Key,-12} : {kayit.Value}");

// Daha kisa yazim
foreach (var (urun, sayi) in stok)
    Console.WriteLine($"{urun} -> {sayi}");
```

Kod 5.4 çıktısı

```
Klavye stogu: 12
Webcam stokta yok
Klavye       : 12
Mouse        : 25
Kulaklik     : 8
```

5.4 Diğer koleksiyonlar

Tablo 5.1 — Koleksiyon seçim rehberi

Tip	Özelliği	Tipik kullanım
`T[] (Array)`	Sabit boyut, en hızlı erişim	Boyutu baştan bilinen veriler
`List<T>`	Değişken boyut, indisli erişim	**Varsayılan tercih**
`Dictionary<K,V>`	Anahtarla çok hızlı erişim	Ürün kodu → ürün eşlemesi
`HashSet<T>`	Tekrarsız eleman kümesi	Benzersiz e-posta listesi
`Queue<T>`	FIFO — ilk giren ilk çıkar	İş kuyruğu, sıra yönetimi
`Stack<T>`	LIFO — son giren ilk çıkar	Geri al (undo) işlemi

Kod 5.5 — HashSet, Queue ve Stack

```
HashSet<string> mailler = ["a@x.com", "b@x.com", "a@x.com"];
Console.WriteLine(mailler.Count);    // 2 -> tekrar eden otomatik elendi

Queue<string> sira = new Queue<string>();
sira.Enqueue("Ali");                 // kuyruga ekle
sira.Enqueue("Veli");
Console.WriteLine(sira.Dequeue());   // Ali  -> ilk giren ilk cikar

Stack<string> gecmis = new Stack<string>();
gecmis.Push("sayfa1");               // yigina ekle
gecmis.Push("sayfa2");
Console.WriteLine(gecmis.Pop());     // sayfa2 -> son giren ilk cikar
```

Haftanın Özeti

1. .NET, **runtime + BCL + SDK** üçlüsünden oluşan çapraz platform bir ekosistemdir. C# kodu önce **IL**'e, çalışma anında **JIT** ile makine diline çevrilir.
2. **.NET 10**, 11 Kasım 2025'te yayımlanan güncel **LTS** sürümdür; C# 14 ve Visual Studio 2026 ile birlikte gelir.
3. Visual Studio'da **solution** birden çok **project**'i kapsar. **Breakpoint** + **F10 / F11** ikilisi hata ayıklamanın temelidir.
4. **Value type** veriyi, **reference type** adresi tutar. Bu ayrım kopyalama davranışını belirler ve OOP konularının temelidir.
5. Para hesaplarında `decimal`, kullanıcı girdisi dönüşümünde `TryParse` kullanılır.
6. `switch expression` ve **pattern matching**, uzun `if-else` zincirlerinin modern alternatifidir.
7. Boyutu değişen veri için `List<T>`, anahtarla erişim için `Dictionary<K,V>`, tekrarsızlık için `HashSet<T>` seçilir.

Anahtar Terimler Sözlüğü

Terim (EN)	Türkçe	Kısa tanım
runtime	çalışma zamanı	Derlenmiş kodu çalıştıran ortam (CLR)
compiler	derleyici	Kaynak kodu IL'e çeviren araç (Roslyn)
garbage collector	çöp toplayıcı	Kullanılmayan belleği otomatik serbest bırakan mekanizma
value type	değer tipi	Verinin kendisini tutan tip (int, struct)
reference type	referans tipi	Verinin adresini tutan tip (class, string)
stack / heap	yığın / öbek	İki ayrı bellek bölgesi
casting	tip dönüşümü	Bir tipi başka bir tipe çevirme
immutable	değiştirilemez	Oluşturulduktan sonra içeriği değişmeyen nesne
generic	genel tip	Tip parametresi alan yapı: List<T>
pattern matching	örüntü eşleme	Değerin şekline göre dallanma
breakpoint	kesme noktası	Programı belirli satırda durduran işaret
LTS	uzun dönem destek	Üç yıl güncelleme alan sürüm

Alıştırmalar

1. Kullanıcıdan iki sayı alıp dört işlem sonucunu yazdıran bir program yazın. Sıfıra bölme durumunu kontrol edin.
2. Kullanıcıdan 5 öğrenci notu alıp ortalamayı, en yüksek ve en düşük notu ekrana yazdıran programı `List<int>` kullanarak yazın.
3. Girilen bir sayının faktöriyelini `for` döngüsüyle hesaplayın. 20'den büyük sayılarda `int` yetmeyeceği için `long` kullanın.
4. 1 ile 100 arasındaki sayıları yazdırın; 3'e bölünenlerde “Fizz”, 5'e bölünenlerde “Buzz”, ikisine birden bölünenlerde “FizzBuzz” yazsın. (Klasik FizzBuzz)
5. Bir `Dictionary<string, decimal>` ile basit bir ürün-fiyat listesi oluşturun. Kullanıcı ürün adı girdiğinde fiyatı göstersin; ürün yoksa uygun bir mesaj versin. Mutlaka `TryGetValue` kullanın.
6. Bir metin içindeki sesli harf sayısını bulan programı `foreach` ile yazın.
7. **Araştırma:** `7 / 2` neden `3` veriyor da `7 / 2.0` `3.5` veriyor? Kendi cümlelerinizle açıklayın.

Ekstra Okuma ve Kaynaklar

- **Microsoft Learn — C# Belgeleri (Türkçe)**
  Resmî C# dokümantasyonunun Türkçe sürümü. Her konu için başvuru kaynağınız burası olmalı.
  https://learn.microsoft.com/tr-tr/dotnet/csharp/
- **C# ile Programlamaya Giriş — Ücretsiz interaktif kurs**
  Tarayıcıdan kod yazarak ilerleyen, kurulum gerektirmeyen resmî Microsoft eğitim yolu.
  https://learn.microsoft.com/tr-tr/training/paths/get-started-c-sharp-part-1/
- **.NET 10 — Yenilikler**
  Runtime, kütüphane ve SDK tarafındaki tüm değişikliklerin resmî listesi.
  https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview
- **C# 14 — Dil Yenilikleri**
  Extension members, field keyword, null-conditional assignment ve diğerleri.
  https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/csharp-14
- **C# Dil Referansı — Built-in tipler**
  Tüm temel veri tiplerinin boyutları, aralıkları ve varsayılan değerleri.
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types
- **C# Kodlama Kuralları (Coding Conventions)**
  Microsoft'un resmî isimlendirme ve biçimlendirme rehberi. Ödevlerde bu kurallara uyulması beklenir.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions
- **Visual Studio 2026 — Hata Ayıklama Rehberi**
  Breakpoint, Watch, Call Stack ve diğer debugging araçlarının adım adım anlatımı.
  https://learn.microsoft.com/en-us/visualstudio/debugger/debugger-feature-tour
- **.NET Fiddle — Tarayıcıda C# çalıştırma**
  Kurulum yapmadan hızlıca kod denemek için. Derste gösterilen örnekleri anında test edebilirsiniz.
  https://dotnetfiddle.net/
- **W3Schools — C# Tutorial**
  Konu başlıklarına göre kısa örnekler ve “Try it Yourself” editörü. Hızlı tekrar için uygundur.
  https://www.w3schools.com/cs/